High Tech Marketing Text Analysis

Brand: Motorola Razr 2025

Group 2

Introduction

Technology fanatics on the internet have been known to be quite vocal of their opinions, this usually takes shape when new smartphone's, gaming consoles, or computers and their components are released. These vocal opinions have especially been the case for foldable smartphones, wherein the new Motorola RAZR's have been notable. Being a revival of the old flip phone model, many individuals could have an emotional attachment to the model and as such verbally share their thoughts on the new phone's.

Sample Selection

This report shall be conducting a text analysis to gauge the consumer sentiment of the new RAZR. In order to ensure that various diverse perspectives are considered for the sentiment surrounding the RAZR revival multiple sources of texts have been captured. This prevents the risk of an echo-chamber like environment of impacting the data collected and ultimately skewing the analysis results.

Data Sources

The data used for the analysis have been gathered from two technology dominant environments on the internet: Reddit and YouTube. From Reddit threads which include the "Motorola Razr 2025" query was used under the notion that it looks at threads which discuss the 2025 RAZR. Then for YouTube comments were gathered from well known tech content creators (such as Marques Brownlee) who reviewed the 2025 RAZR.

Sampling representation & Bias

It must be acknowledged that there is a risk of an inherent bias being introduced by choosing to look at both these specific platforms, and user base. Of the large population of individuals who own smartphone's many are not active on Reddit and even with regards to YouTube which does have a larger install base, many are still not actively interacting (posting or commenting). Individuals who are passionate or involved enough to do either of the aforementioned actions could also be far more critical than the average consumer. Though the case may also be made that it is these exact tech fanatics who are active with the newest technologies who would be the consumers for such a commodity as a folding smartphone.

Data Collection

Using the code chunk below Reddit is scraped for data which includes certain key words which aid in the gathering of posts as well as comments relating to the RAZR foldable. Key words such as "Android" and

"Motorola" are also crucial in ensuring that the data doesn't contain either products from other companies which may also be called close to RAZR (e.g. scooters) or even literal razors.

```
fetch_reddit_thread <- function(thread_url, limit = 500, attempts = 5) {
  api_url <- paste0(thread_url, ".json?limit=", limit)
  response <- RETRY(
    "GET",
    api_url,
    user_agent(paste0("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) ",
                      "AppleWebKit/537.36 (KHTML, like Gecko) ",
                      "Chrome/125.0.0.0 Safari/537.36")),
    times        = attempts,
    pause_base   = 2,
    pause_cap    = 20,
    terminate_on = c(400, 401, 403, 404)
  )

  if (http_error(response)) {
    stop("HTTP ", status_code(response), " when fetching ", api_url)
  }

  payload <- fromJSON(
    content(response, as = "text", encoding = "UTF-8"),
    simplifyVector = FALSE
  )
  post <- payload[[1]]$data$children[[1]]$data

  flatten_comments <- function(children) {
    if (!is.list(children)) return(data.frame())
    rows <- list()
    for (child in children) {
      if (!is.null(child$kind) && child$kind == "t1") {
        comment   <- child$data
        body_text <- comment$body %||% NA_character_
        if (!is.null(body_text)) body_text <- gsub("[\r\n]+", " ", body_text)
        rows[[length(rows) + 1]] <- data.frame(
          type      = "comment",
          id        = comment$id        %||% NA_character_,
          parent_id = comment$parent_id %||% NA_character_,
          author    = comment$author    %||% NA_character_,
          body      = body_text,
          score     = comment$score     %||% NA_real_,
          subreddit = comment$subreddit %||% NA_character_,
          stringsAsFactors = FALSE
        )
        replies <- if (is.list(comment$replies) &&
                       !is.null(comment$replies$data$children))
          comment$replies$data$children else list()
        if (length(replies) > 0)
          rows <- c(rows, flatten_comments(replies))
      }
    }
    if (length(rows) == 0) return(data.frame())
    rows <- Filter(function(x) is.data.frame(x) && ncol(x) > 0, rows)
```

```r
    if (length(rows) == 0) return(data.frame())
    tryCatch(
      dplyr::bind_rows(rows),
      error = function(e) {
        cols     <- c("type","id","parent_id","author","body","score","subreddit")
        repaired <- lapply(rows, function(r) {
          if (!is.data.frame(r)) return(NULL)
          for (m in setdiff(cols, names(r))) r[[m]] <- NA
          r[cols]
        })
        dplyr::bind_rows(Filter(Negate(is.null), repaired))
      }
    )
  }

  comments  <- flatten_comments(payload[[2]]$data$children %||% list())
  post_body <- post$selftext %||% post$title %||% NA_character_
  if (!is.null(post_body)) post_body <- gsub("[\r\n]+", " ", post_body)

  post_df <- data.frame(
    type      = "post",
    id        = post$id     %||% NA_character_,
    parent_id = NA_character_,
    author    = post$author %||% NA_character_,
    body      = post_body,
    score     = post$score  %||% NA_real_,
    subreddit = post$subreddit %||% NA_character_,
    stringsAsFactors = FALSE
  )

  if (!is.data.frame(comments)) comments <- data.frame()
  tryCatch(
    dplyr::bind_rows(post_df, comments),
    error = function(e) post_df
  )
}

suppressWarnings({
  reddit_cache <- "data/reddit_content.csv"
  cached <- file.exists(reddit_cache)

  if (cached) {
    reddit_all <- tryCatch(
      read.csv(reddit_cache, stringsAsFactors = FALSE),
      error = function(e) { message("Cache read failed: ", conditionMessage(e)); data.frame() }
    )
  } else {
    search_config <- list(
      list(sub = "Android",       kw = "motorola razr"),
      list(sub = "Android",       kw = "razr 2025"),
      list(sub = "Android",       kw = "razr flip"),
      list(sub = "Motorola",      kw = "razr"),
      list(sub = "Motorola",      kw = "razr ultra"),
```

```r
      list(sub = "Motorola",      kw = "razr plus"),
      list(sub = "phones",        kw = "razr"),
      list(sub = "phones",        kw = "motorola"),
      list(sub = "smartphones",   kw = "motorola razr"),
      list(sub = "razr",          kw = "razr"),
      list(sub = "razr",          kw = "motorola"),
      list(sub = "technology",    kw = "motorola razr"),
      list(sub = "technology",    kw = "razr 2025")
    )

    reddit_posts <- dplyr::bind_rows(Filter(
      function(x) is.data.frame(x) && nrow(x) > 0,
      lapply(search_config, function(cfg) {
        result <- tryCatch({
          Sys.sleep(2)
          find_thread_urls(keywords = cfg$kw, subreddit = cfg$sub, sort_by = "top")
        }, error = function(e) {
          message("Failed r/", cfg$sub, " / '", cfg$kw, "': ", conditionMessage(e))
          data.frame(url = character())
        })
        if (!is.data.frame(result)) return(data.frame(url = character()))
        result
      })
    )) |> dplyr::distinct(url, .keep_all = TRUE)

    reddit_max_threads <- 250

    if (nrow(reddit_posts) > 0) {
      to_fetch <- head(reddit_posts$url, reddit_max_threads)
      raw_list <- lapply(seq_along(to_fetch), function(i) {
        Sys.sleep(1)
        tryCatch(
          fetch_reddit_thread(to_fetch[i]),
          error = function(e) { message("Failed: ", conditionMessage(e)); data.frame() }
        )
      })
      reddit_all <- dplyr::bind_rows(
        Filter(function(x) is.data.frame(x) && nrow(x) > 0, raw_list)
      )
    } else {
      reddit_all <- data.frame()
    }

    if (!dir.exists("data")) dir.create("data")
    write.csv(reddit_all, reddit_cache, row.names = FALSE)
  }
})

cat("Reddit total rows:", nrow(reddit_all), "\n")
```

```
## Reddit total rows: 992
```

```
cat("Posts vs comments:\n");  print(table(reddit_all$type))
```

```
## Posts vs comments:

##
## comment    post
##     765     227
```

```
cat("By subreddit:\n");       print(table(reddit_all$subreddit))
```

```
## By subreddit:

##
##     Android    motorola      phones        razr Smartphones
##           6         312          29         608          37
```

The same is done for YouTube in the code chunk below. Though it is somewhat simpler, given that unlike Reddit where a plethora of posts and comments must be scraped individually, on YouTube only comments for specific known video's are observed. The selected video's themselves correspond with content creators who have a reputation for being trustworthy with unbiased reviews.

```
fetch_youtube_comments <- function(video_id, max_comments = 400, api_key = Sys.getenv("YOUTUBE_KEY")) {
  if (identical(api_key, "")) {
    stop("YOUTUBE_KEY is not set. Load it from .env or your environment before knitting.")
  }

  all_comments <- list()
  page_token   <- NULL

  repeat {
    query <- list(
      part       = "snippet",
      videoId    = video_id,
      maxResults = 100,
      textFormat = "plainText",
      key        = api_key
    )
    if (!is.null(page_token)) query$pageToken <- page_token

    resp <- httr::GET("https://www.googleapis.com/youtube/v3/commentThreads",
                      query = query)

    if (httr::http_error(resp)) {
      error_body <- tryCatch(httr::content(resp, as = "text", encoding = "UTF-8"),
                             error = function(e) "")
      message("YouTube API error: ", httr::status_code(resp),
              if (nzchar(error_body)) paste0(" - ", error_body) else "")
      break
    }

    parsed <- jsonlite::fromJSON(
```

```r
      httr::content(resp, as = "text", encoding = "UTF-8"),
      flatten = TRUE
    )

    items <- parsed$items
    if (is.null(items) || nrow(items) == 0) break

    batch <- data.frame(
      comment_id = items$id,
      author     = items$snippet.topLevelComment.snippet.authorDisplayName,
      body       = items$snippet.topLevelComment.snippet.textDisplay,
      like_count = items$snippet.topLevelComment.snippet.likeCount,  # kept for segmentation
      stringsAsFactors = FALSE
    )

    all_comments[[length(all_comments) + 1]] <- batch
    collected <- sum(sapply(all_comments, nrow))

    page_token <- parsed$nextPageToken
    if (is.null(page_token) || collected >= max_comments) break
    Sys.sleep(0.5)
  }

  if (length(all_comments) == 0) return(data.frame())
  dplyr::bind_rows(all_comments) |> dplyr::mutate(source = "youtube")
}

razr_videos <- list(
  mkbhd      = "8om1eJrO21U",   # MKBHD Razr 2025 review
  dave2d     = "JTT67FSc8j8",   # ShortCircuit review
  mrmobile   = "YGTkjchlVJk",   # MrMobile review
  phonearena = "b3Ijuf7DHcQ"    # PhoneArena review
)

if (file.exists("data/youtube_comments.csv")) {
  youtube_all <- read.csv("data/youtube_comments.csv", stringsAsFactors = FALSE)
  cat("Loaded YouTube comments from CSV:", nrow(youtube_all), "\n")
} else {
  youtube_all <- purrr::map2_dfr(
    razr_videos,
    names(razr_videos),
    ~ fetch_youtube_comments(.x, max_comments = 400) |>
        dplyr::mutate(channel = .y)
  )
  if (!dir.exists("data")) dir.create("data")
  write.csv(youtube_all, "data/youtube_comments.csv", row.names = FALSE)
}
```

```
## Loaded YouTube comments from CSV: 1161
```

```r
cat("YouTube comments collected:", nrow(youtube_all), "\n")
```

```
## YouTube comments collected: 1161
```

```r
print(table(youtube_all$channel))
```

```
##
##     dave2d      mkbhd   mrmobile phonearena
##        400        400        257        104
```

Exploration

With the initial scraping completed, attention may now be given to the raw data which has been collected. While it may not yet provide any useful conclusions, it can already provide some insight on the split between the collected data. While there is a discrepancy between the raw corpus volumes, it is not a magnitude which would be of concern in our view.

```r
exploration_summary <- tibble::tibble(
  Source    = c("Reddit (comments only)", "YouTube"),
  Documents = c(sum(reddit_all$type == "comment", na.rm = TRUE),
                nrow(youtube_all)),
  Avg_chars = c(round(mean(nchar(reddit_all$body[reddit_all$type == "comment"]), na.rm = TRUE), 1),
                round(mean(nchar(youtube_all$body), na.rm = TRUE), 1))
)
knitr::kable(exploration_summary, caption = "Raw corpus volume by source")
```

Table 1: Raw corpus volume by source

Source	Documents	Avg_chars
Reddit (comments only)	765	218.1
YouTube	1161	153.8

Pre-Processing

As was mentioned previously, the data currently exists in two separate data frames; the Reddit and YouTube data. In the code chunk below these two are merged to create one large database to be used for further text analyses. In order to still maintain some tractability between the data a new column shall be added to indicate which source the data was collected from and then binding them together. Furthermore an overview may be provided for the languages comments are left in, which shall be filtered and removed accordingly later on.

```
##
##      de      en      es      fr      pt      ro      ru xx-Qaai
##       1    1792       7       1       2       1       1       1
```

Filtering & Cleaning

The data may now be filtered by various means. From maintaining only data in English, removing empty data points, to removing non-ASCII characters, the data balanced is slightly shifted but not to concerning bounds.

```r
# Keep English only
combined_data <- combined_data |> filter(language == "en")

# Strip non-ASCII characters
combined_data$body <- gsub("[^\x01-\x7F]", "", combined_data$body)

# Remove now-empty bodies
combined_data <- combined_data |> filter(body != "")

# Old vs. New era
# Flag documents that reference the original
old_terms <- c("original", "old razr", "2004", "2005", "v3",
               "flip phone days", "back in the day", "nostalgia",
               "first razr", "classic", "used to have", "the old one")
combined_data <- combined_data |>
  mutate(era = ifelse(
    grepl(paste(old_terms, collapse = "|"), body, ignore.case = TRUE),
    "references_old", "new_only"))

cat("Era distribution (old-brand references vs new-only):\n")
```

```
## Era distribution (old-brand references vs new-only):
```

```r
print(table(combined_data$era))
```

```
##
##       new_only references_old
##           1753             39
```

```r
cat("\nDocuments by platform:\n")
```

```
##
## Documents by platform:
```

```r
print(table(combined_data$platform))
```

```
##
##  Reddit YouTube
##     682    1110
```

Modeling

With the collected data now filtered and cleaned up, work may be done on the actual modelling of the text analysis.

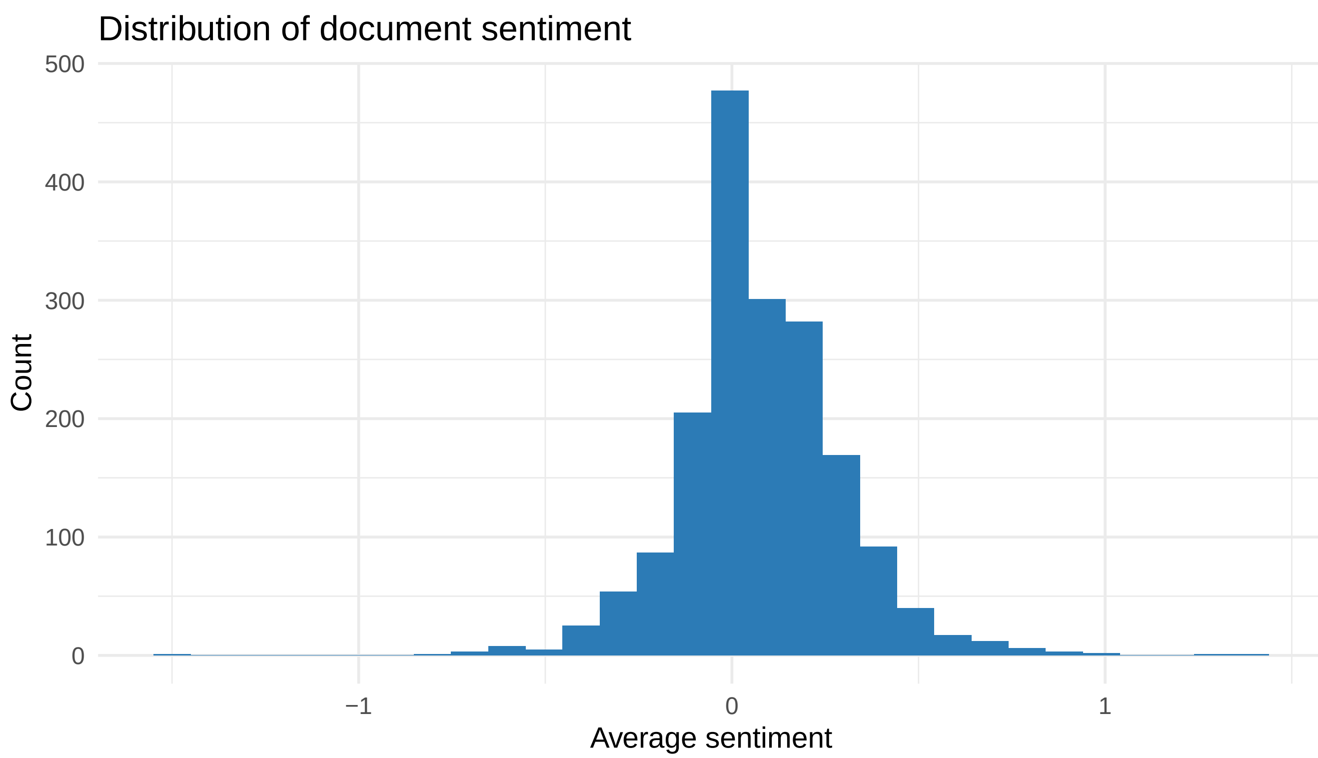
Corpus Creation

The data may now be used to create a corpus, with this the field is spread out around the individual reviews and their subsequent sentiment towards the RAZR.

```r
corpus <- quanteda::corpus(combined_data, docid_field = "id", text_field = "body")

quanteda::docvars(corpus, field = "ave_sentiment") <-
  sentimentr::sentiment_by(sentimentr::get_sentences(corpus))$ave_sentiment

# Distribution of sentiment across the corpus
ggplot(data.frame(s = quanteda::docvars(corpus)$ave_sentiment),
       aes(x = s)) +
  geom_histogram(bins = 30, fill = "#2C7BB6") +
  labs(title = "Distribution of document sentiment",
       x = "Average sentiment", y = "Count") +
  theme_minimal()
```



Tokenization

With the corpus created the data goes though tokenization, with this the text is broken down into smaller pieces by way of removing aspects such as punctuation, separators, and other symbols as may be seen in the chunk below. This will allow the text to be numerically analyzed further down the line.

```r
tokens <- quanteda::tokens(corpus,
                           what = "word",
                           remove_punct = TRUE,
                           remove_symbols = TRUE,
                           remove_numbers = TRUE,
                           remove_url = TRUE,
                           remove_separators = TRUE,
```

```r
                           split_hyphens = FALSE,
                           split_tags = FALSE,
                           include_docvars = TRUE,
                           padding = FALSE,
                           verbose = FALSE)

tokens <- quanteda::tokens_tolower(tokens)
```

Synonym consolidation

Now the tokens are also consolidated based on if they are synonyms of other collected words. This aids in getting a clearer consensus of the items which really matter for the consumers, since a camera is just important as a cam.

```r
tokens <- quanteda::tokens_replace(tokens,
  pattern = c(
    "foldable", "folding",
    "flipping",
    "cam", "cams",
    "charging", "charger",
    "displays",
    "hinges", "hinging",
    "pricing", "priced", "pricey", "expensive", "costly",
    "galaxy",
    "ios", "apple"
  ),
  replacement = c(
    "fold", "fold",
    "flip",
    "camera", "camera",
    "charge", "charge",
    "display",
    "hinge", "hinge",
    "price", "price", "price", "price", "price",
    "samsung",
    "apple", "apple"
  ),
  valuetype = "fixed")
```

Stop Words Removal

The data still includes a plethora of words which work to provide structure to sentences, but in the context of data modelling serve little to no purpose. These stop words are removed accordingly, as well as certain key words which were required to obtain the data specifically relating to the Motorola RAZR but serve little purpose now as well.

```r
custom_stops <- c(
  "razr", "motorola", "phone", "just", "get", "like", "thing", "really",
  "know", "time", "people", "would", "also", "much", "use", "see",
  "one", "will", "can", "even", "still", "want", "review",
  "moto", "device", "smartphone", "im", "dont", "ive", "thats", "got",
```

```r
  "amp", "u", "ur", "ok", "yeah", "lol", "gonna", "good", "make",
  "think", "need", "look", "come", "say"
)
tokens <- quanteda::tokens_remove(
  tokens, pattern = c(quanteda::stopwords("en"), custom_stops))
```

Stemming & Lemmatization

Having simplified and consolidated the words together, they may now be broken down to their root term by way of stemming and lemmatization. This ensures that small variations of words will be treated as the same term.

```r
feats <- quanteda::types(tokens)
tokens <- quanteda::tokens_replace(
  tokens,
  pattern     = feats,
  replacement = textstem::lemmatize_words(feats),
  valuetype   = "fixed")
```

DFM Creation & Frequency Filtering

A Document-Feature-Matrix (DFM) may now be made with the stemmed and lemmatized tokens, here the rows shall be the documents, and the columns shall be unique words. With this some additional filtering steps are applied to clean the matrix from rare, empty, and large terms are removed. This reduces the features significantly with the final FDM having 559 features by 1737 documents.

```r
dfm <- quanteda::dfm(tokens)

# Keep only tokens of 3+ characters
dfm <- quanteda::dfm_select(dfm, pattern = "^.{3,}$",
                            valuetype = "regex", selection = "keep")

cat("Features before frequency trim:", nfeat(dfm), "\n")
```

```
## Features before frequency trim: 4052
```

```r
# Drop very rare terms (appear fewer than 10)
dfm <- dfm_trim(dfm, min_termfreq = 10)
cat("Features after frequency trim:", nfeat(dfm), "\n")
```

```
## Features after frequency trim: 559
```

```r
cat("\nMost frequent features:\n")
```

```
##
## Most frequent features:
```

```r
print(topfeatures(dfm, 15))
```

```
##  screen    fold    good    flip   phone   ultra samsung    year     app     buy
##     424     265     242     215     201     201     181     179     173     168
##     use  camera    love    work     now
##     164     162     156     138     136
```

```r
# Drop any documents left empty after all filtering
empty_docs <- which(rowSums(dfm) == 0)
if (length(empty_docs) > 0) dfm <- dfm[-empty_docs, ]
cat("\nFinal DFM:", ndoc(dfm), "documents x", nfeat(dfm), "features\n")
```

```
##
## Final DFM: 1737 documents x 559 features
```

Selecting K

Prior to the topic modelling the split of data used for training and testing must be determined, for this the ideal number of topics or k's must be selected. This is done with the elbow method, where perplexity is computed from a Document Term Matrix (DTM) from a converted DFM. From this a k of seven is selected as its the sharpest elbow visible.

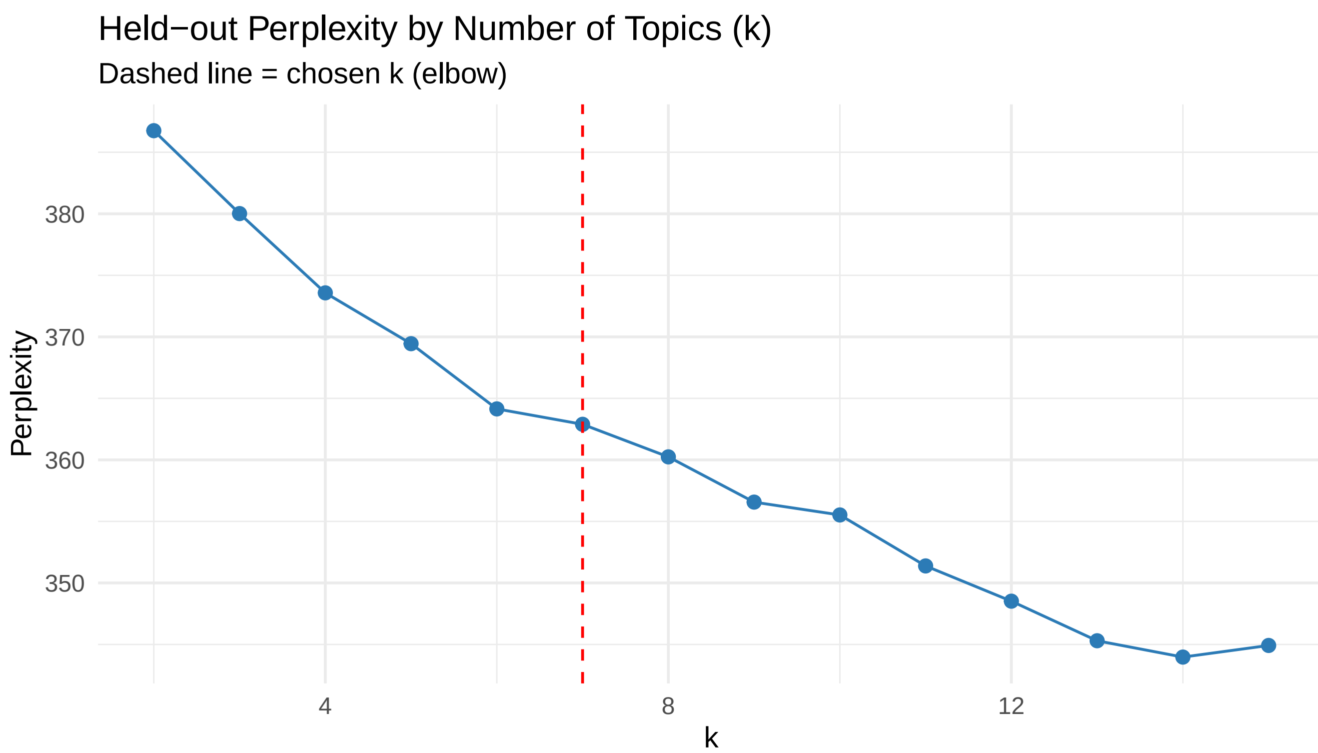
```r
dtm <- quanteda::convert(dfm, to = "topicmodels")

set.seed(42)
n_docs    <- nrow(dtm)
train_idx <- sample(n_docs, floor(n_docs * 0.8))
dtm_train <- dtm[train_idx, ]
dtm_test  <- dtm[-train_idx, ]

k_values <- 2:15
perplexity_scores <- sapply(k_values, function(k) {
  m <- topicmodels::LDA(dtm_train, k = k, method = "Gibbs",
                        control = list(seed = 42, iter = 1000, burnin = 200))
  topicmodels::perplexity(m, newdata = dtm_test)
})

perplexity_df <- data.frame(k = k_values, perplexity = perplexity_scores) |>
  mutate(improvement = dplyr::lag(perplexity) - perplexity)

ggplot(perplexity_df, aes(x = k, y = perplexity)) +
  geom_line(colour = "#2C7BB6") +
  geom_point(colour = "#2C7BB6", size = 2) +
  geom_vline(xintercept = 7, linetype = "dashed", colour = "red") +
  labs(title = "Held-out Perplexity by Number of Topics (k)",
       subtitle = "Dashed line = chosen k (elbow)",
       x = "k", y = "Perplexity") +
  theme_minimal()
```




```r
knitr::kable(perplexity_df, digits = 2,
             caption = "Held-out perplexity and per-step improvement by k")
```

Table 2: Held-out perplexity and per-step improvement by k

k	perplexity	improvement
2	386.75	NA
3	380.02	6.74
4	373.57	6.44
5	369.44	4.13
6	364.14	5.30
7	362.89	1.25
8	360.25	2.65
9	356.57	3.68
10	355.53	1.04
11	351.38	4.14
12	348.52	2.86
13	345.30	3.22
14	343.97	1.33
15	344.92	-0.95

LDA

The k of 7 is then used within a Latent Dirichlet Allocation (LDA) topic model, here the data is assigned to their most likely topics from which they are then assigned to an ID. This will serve to identify and match reoccurring words into specific topics.

```r
K <- 7

set.seed(42)
lda_model <- seededlda::textmodel_lda(dfm, k = K)

topic_assignments <- data.frame(
  doc_id    = docnames(dfm),
  topic     = topics(lda_model),
  sentiment = docvars(dfm)$ave_sentiment,
  stringsAsFactors = FALSE
)
```

Stability Analysis

Since LDA results can vary based on the seed which was randomly assigned during the model fitting a stbility analysis is conducted to see how the topics perform over three different runs. For each run the top five words are collected per topic, these are then compiled and shown, though little may be taken away from their content it does show quite some overlap between the seeds.

```r
seeds <- c(42L, 123L, 999L)

stability_results <- lapply(seeds, function(s) {
  set.seed(s)
  m     <- seededlda::textmodel_lda(dfm, k = K)
  trms  <- seededlda::terms(m, 5)
  apply(trms, 2, paste, collapse = ", ")
})

stability_df <- data.frame(
  topic    = paste0("Topic ", 1:K),
  seed_42  = stability_results[[1]],
  seed_123 = stability_results[[2]],
  seed_999 = stability_results[[3]],
  row.names = NULL
)

knitr::kable(stability_df,
             col.names = c("Topic", "Seed 42", "Seed 123", "Seed 999"),
             caption   = "Top 5 words per topic across 3 seeds (stability check)")
```

Table 3: Top 5 words per topic across 3 seeds (stability check)

Topic	Seed 42	Seed 123	Seed 999
Topic 1	screen, fold, case, now, year	year, iphone, android, update, since	year, iphone, samsung, now, update
Topic 2	fold, mine, wait, day, say	charge, video, now, battery, call	app, use, work, mode, google
Topic 3	flip, screen, small, phone, love	screen, small, fold, outside, big	screen, case, break, issue, work
Topic 4	samsung, year, iphone, android, phone	flip, samsung, phone, love, fold	fold, wait, mine, come, say

Topic	Seed 42	Seed 123	Seed 999
Topic 5	good, price, ultra, battery, model	fold, case, wait, mine, come	charge, camera, video, battery, call
Topic 6	camera, app, use, call, play	good, price, ultra, model, great	flip, love, phone, screen, small
Topic 7	app, update, display, good, screen	app, camera, try, work, google	good, ultra, price, samsung, model

Results

Having completed the modelling analyses, the results may now be highlighted. From the individual words or topics, towards their relation to overall consumer sentiment, the results shall show valuable insights on just how the posters feel towards the revived RAZR.

Topic Labels

Firstly seven various topics may be created and assigned with terms which relate closely to said topics. The code chunk below highlights the topics themselves as well as the top 10 most frequent terms which appear per topic.

```r
# Top 10 terms per topic
topic_terms <- seededlda::terms(lda_model, 10)
knitr::kable(topic_terms, caption = "Top 10 terms per topic")
```

Table 4: Top 10 terms per topic

topic1	topic2	topic3	topic4	topic5	topic6	topic7
screen	fold	flip	samsung	good	camera	app
fold	mine	screen	year	price	app	update
case	wait	small	iphone	ultra	use	display
now	day	phone	android	battery	call	good
year	say	love	phone	model	play	screen
issue	come	big	ultra	charge	video	external
break	trade	back	flip	phone	google	thank
buy	buy	outside	apple	new	feature	work
drop	ship	fold	love	plus	watch	use
little	deal	hand	good	great	pixel	try

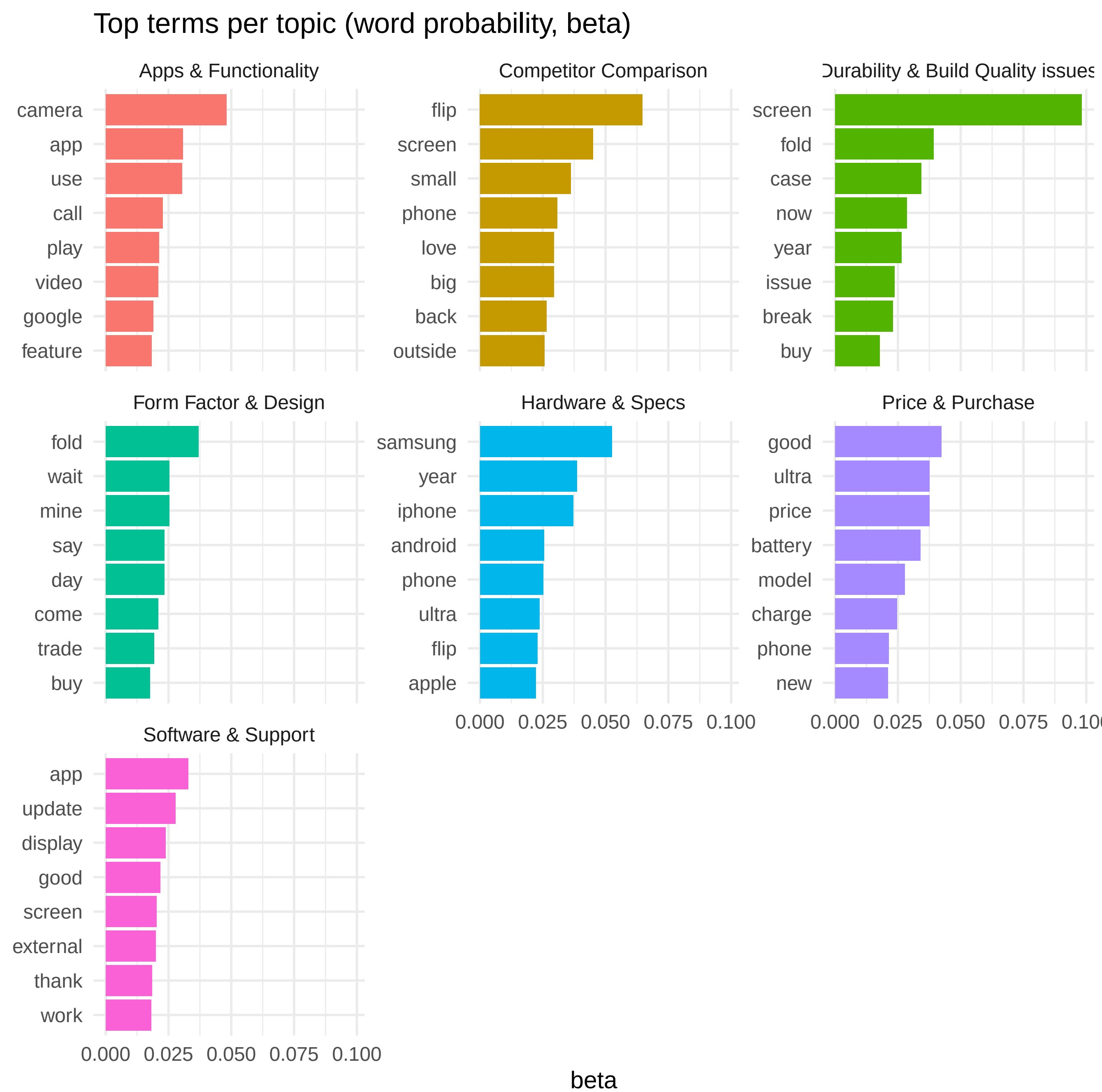
```r
# Labels
topic_labels <- c(
  "topic1" = "Durability & Build Quality issues",
  "topic2" = "Form Factor & Design",
  "topic3" = "Competitor Comparison",
  "topic4" = "Hardware & Specs",
  "topic5" = "Price & Purchase",
  "topic6" = "Apps & Functionality",
  "topic7" = "Software & Support"
)
```

Topic Word Distributions

Continuing with the seven topics the distribution of the top ten terms may now be shown. Rather than just showing them in a ranked order one may now also see just how important the various terms are for the posters. As one might expect the code chunk below shows that a great point of importance for the consumers for the foldable phone is the screen, fold, and flip.

```r
# Extract per-topic word probabilities
phi <- lda_model$phi
beta_df <- as.data.frame(t(phi)) |>
  tibble::rownames_to_column("term") |>
  tidyr::pivot_longer(-term, names_to = "topic", values_to = "beta") |>
  group_by(topic) |>
  slice_max(beta, n = 8, with_ties = FALSE) |>
  ungroup() |>
  mutate(label = topic_labels[topic])

ggplot(beta_df,
       aes(x = tidytext::reorder_within(term, beta, topic),
           y = beta, fill = label)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~label, scales = "free_y") +
  coord_flip() +
  tidytext::scale_x_reordered() +
  labs(title = "Top terms per topic (word probability, beta)",
       x = NULL, y = "beta") +
  theme_minimal()
```

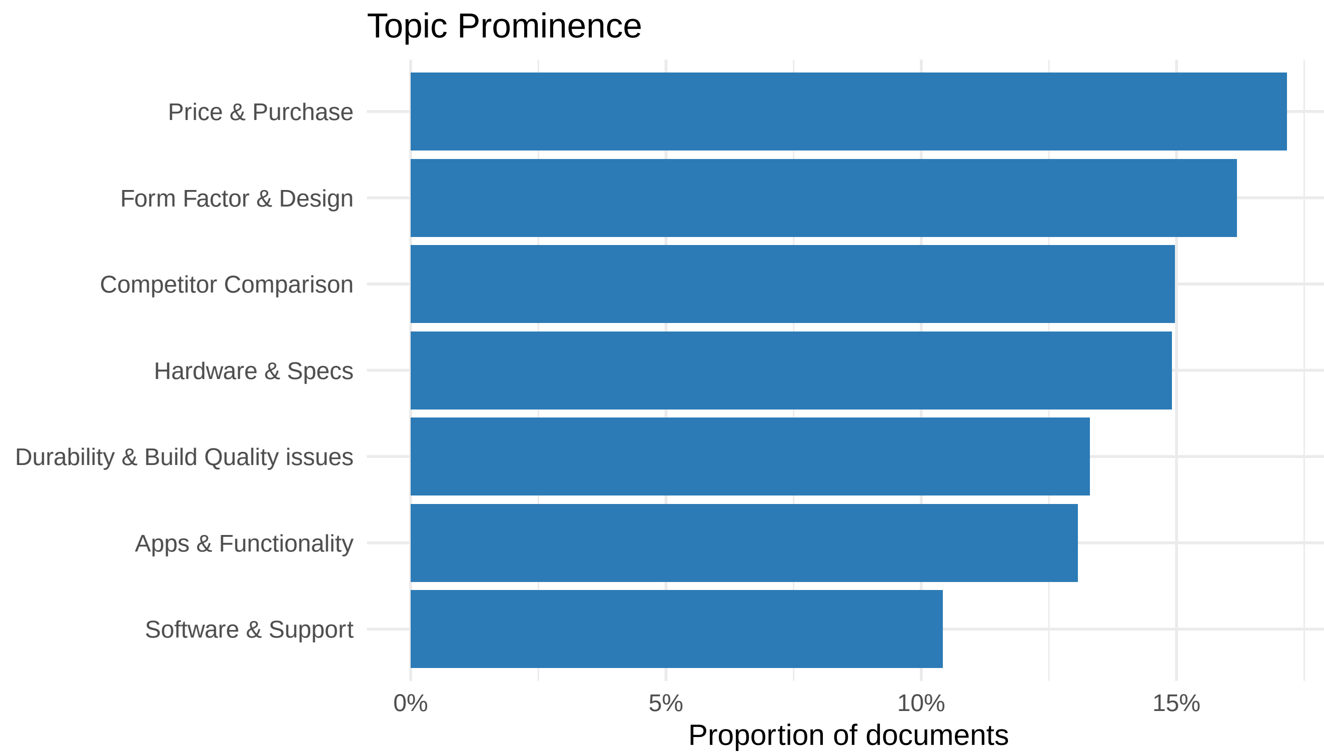


Topic Prominence

Sticking with the seven topics, they are now cross-referenced to each other. This will aid in showing just how widely discussed each of these topics are to the consumers, at least in how prominently they're mentioned. The code chunk below shows that the most prominent topic is for "Apps & Functionality", with the least prominent one being the "Software & Support". This is interesting in showing that the actual functioning of the device is a greater point of discussion, which would make sense given the RAZR would simply run a variant of well established Android OS.

```r
topic_counts <- as.data.frame(table(topics(lda_model))) |>
  rename(topic = Var1, n = Freq) |>
  mutate(prop  = n / sum(n),
         label = topic_labels[as.character(topic)])

ggplot(topic_counts, aes(x = reorder(label, prop), y = prop)) +
  geom_col(fill = "#2C7BB6") +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Topic Prominence",
       x = NULL, y = "Proportion of documents") +
  theme_minimal()
```




```r
knitr::kable(topic_counts |> select(label, n, prop),
             digits = 3, col.names = c("Topic", "N", "Proportion"),
             caption = "Relative prominence of each topic")
```

Table 5: Relative prominence of each topic

Topic	N	Proportion
Durability & Build Quality issues	231	0.133
Form Factor & Design	281	0.162
Competitor Comparison	260	0.150
Hardware & Specs	259	0.149
Price & Purchase	298	0.172

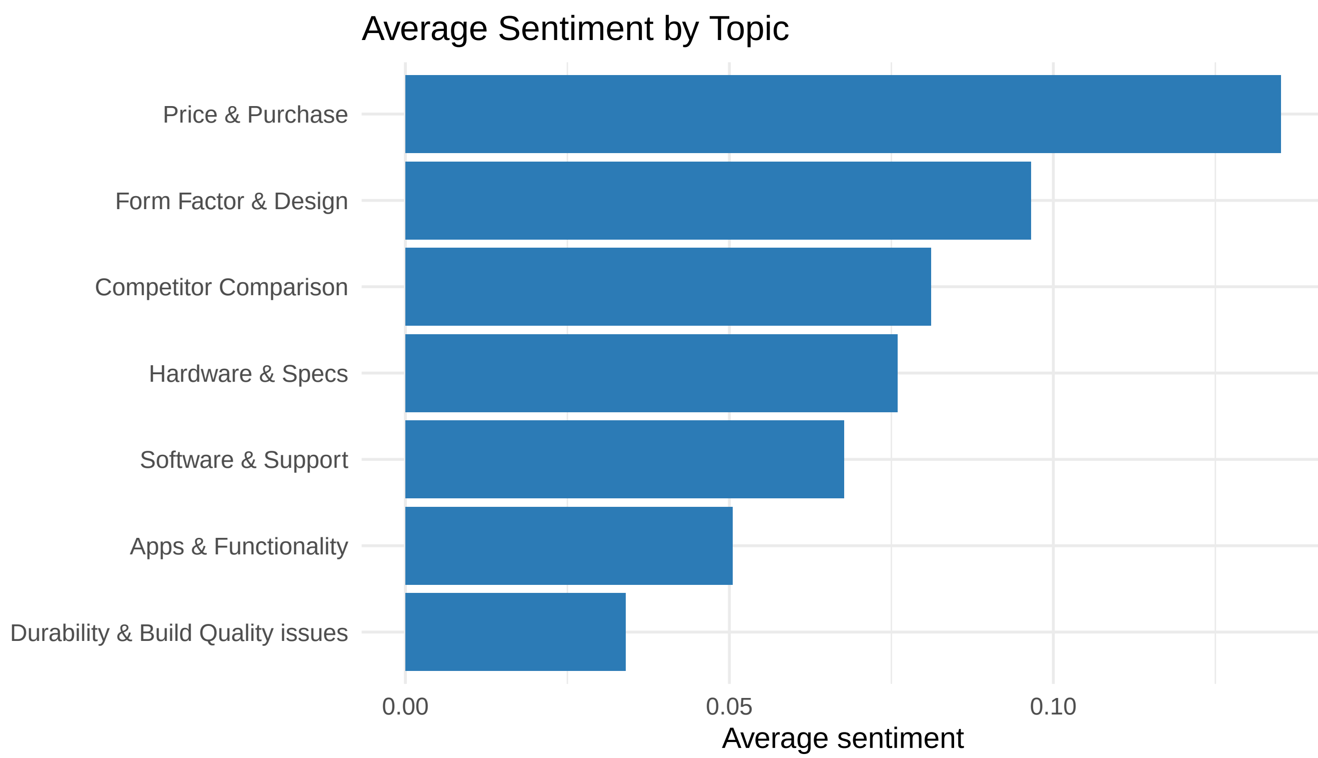
Topic	N	Proportion
Apps & Functionality	227	0.131
Software & Support	181	0.104

Sentiment by Topic

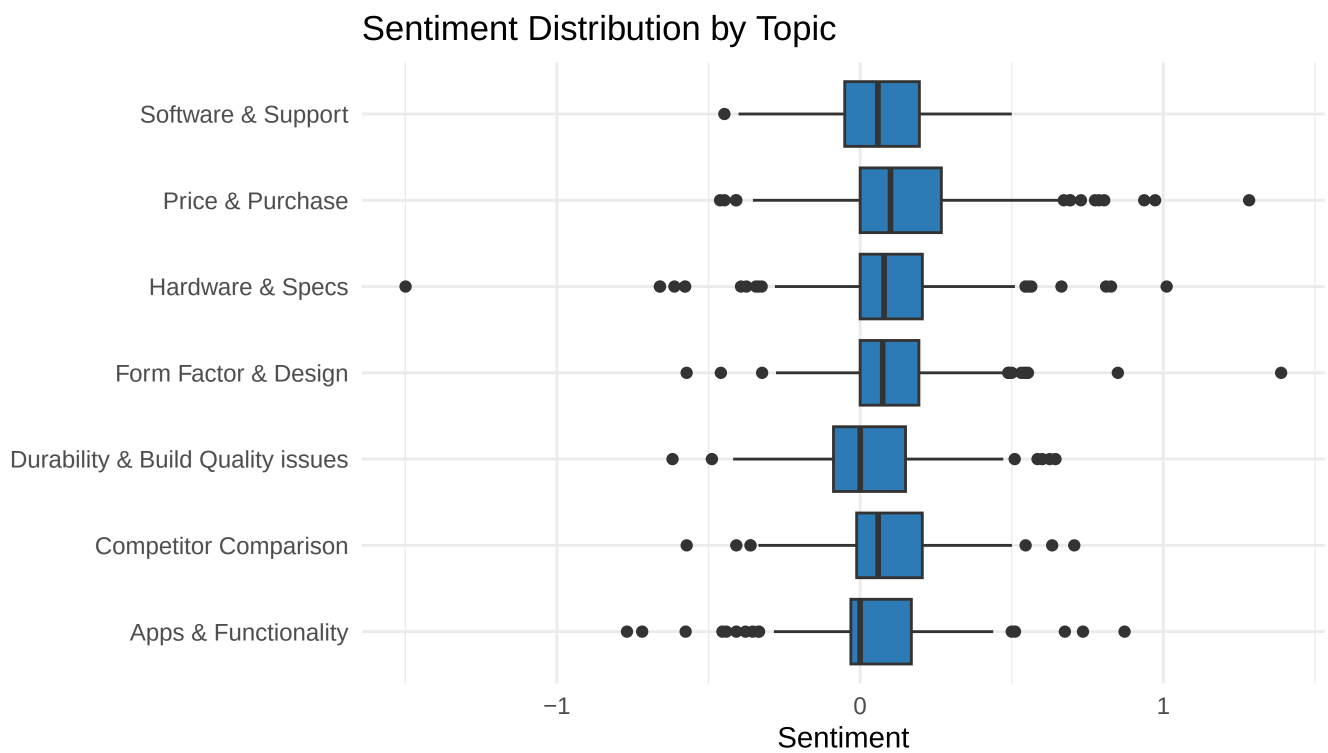
As for the consumer sentiment around each topic, the average sentiment scores are grouped by topic, from which the mean is then calculated. From this a box plot is used to show that the Price & Purchase topic ranks as the highest average sentiment by a solid margin among consumers, meaning that they view the topic quite positively. On the opposite side of the spectrum is the Durability & Build Quality Issues topic, indicating that consumers do not perceive said topic positively. These may be expected, given that the foldable smartphones are known to have good internal specifications, while their primary weakness is, and this might come as a surprise to some, is the foldable discplay. The plastic screen and hinge mechanisms have their kinks and are still perceived as fragile.

```r
topic_sentiment <- topic_assignments |>
  group_by(topic) |>
  summarise(mean_sentiment = mean(sentiment, na.rm = TRUE),
            n = n(), .groups = "drop") |>
  mutate(label = topic_labels[topic])

ggplot(topic_sentiment, aes(x = reorder(label, mean_sentiment), y = mean_sentiment)) +
  geom_col(fill = "#2C7BB6") +
  coord_flip() +
  labs(title = "Average Sentiment by Topic",
       x = NULL, y = "Average sentiment") +
  theme_minimal()
```




```r
ggplot(topic_assignments |> mutate(label = topic_labels[topic]),
       aes(x = label, y = sentiment)) +
  geom_boxplot(fill = "#2C7BB6") +
  coord_flip() +
  labs(title = "Sentiment Distribution by Topic",
       x = NULL, y = "Sentiment") +
  theme_minimal()
```



Statistical Tests

Through much of the data collecting and filtering brief mentions were given on the data split between Reddit and YouTube. These were primarily numerical, but how does their data differ inherently? That is what will be explored below statistically.

Do the two customer segments (Reddit and YouTube) discuss different topics?

With Reddit and YouTube having inherently different user bases, one would assume that their topics of prominence would also differ. The code chunk below serves to illustrate that Reddit is far more interested in software and its comparison to competitors, while YouTube users focus much more on physical aspects such as build quality and hardware specifications. This could be as a result that Reddit is often used for software support, whereas YouTube is used for real world tutorials.

```r
topic_source <- topic_assignments |>
  mutate(doc_id = as.integer(doc_id)) |>
  left_join(combined_data |> select(id, platform, era),
            by = c("doc_id" = "id")) |>
  mutate(label = topic_labels[topic])

contingency <- table(topic_source$label, topic_source$platform)
knitr::kable(contingency, caption = "Topic distribution: Reddit vs YouTube")
```

Table 6: Topic distribution: Reddit vs YouTube

	Reddit	YouTube
Apps & Functionality	85	148
Competitor Comparison	32	223
Durability & Build Quality issues	95	135
Form Factor & Design	179	104
Hardware & Specs	70	185
Price & Purchase	108	189
Software & Support	101	83

```r
chi_result <- chisq.test(contingency)
print(chi_result)
```

```
##
##  Pearson's Chi-squared test
##
## data:  contingency
## X-squared = 181.38, df = 6, p-value < 2.2e-16
```

Does sentiment differ across topics?

A Kruskal-Wallis test may be used to check whether the sentiment scores differ across the seven topics, from which a Wilcoxon rank-sum tests are run between the topic pairs with an adjusted p-value using a Benjamini-Hochberg (BH) method. The results of which show that the tests are significant, meaning that the sentiment does differ between the topics.

```r
kw_result <- kruskal.test(sentiment ~ factor(topic), data = topic_assignments)
print(kw_result)
```

```
##
##  Kruskal-Wallis rank sum test
##
## data:  sentiment by factor(topic)
## Kruskal-Wallis chi-squared = 29.963, df = 6, p-value = 3.994e-05
```

```r
pw <- pairwise.wilcox.test(topic_assignments$sentiment,
                           topic_assignments$topic,
                           p.adjust.method = "BH")
print(pw)
```

```
##
##  Pairwise comparisons using Wilcoxon rank sum test with continuity correction
##
## data:  topic_assignments$sentiment and topic_assignments$topic
##
##        topic1  topic2 topic3 topic4 topic5 topic6
## topic2 0.0042  -      -      -      -      -
## topic3 0.0259  0.7545 -      -      -      -
## topic4 0.0259  0.7817 0.9250 -      -      -
```

```
## topic5 8.2e-05 0.1491 0.0906 0.0906 -      -
## topic6 0.4568  0.0423 0.0906 0.0906 0.0012 -
## topic7 0.0906  0.4880 0.7054 0.7054 0.0522 0.3298
##
## P value adjustment method: BH
```

Old vs New Era: does sentiment differ?

Much has been discussed on user sentiments and prominence on the outlined seven topics, however some insight may also be found in comparing the old RAZR models to the new ones. Two points should be stressed. First, the old-referencing group is very small (39 documents against 1,698), so any comparison is tentative. Second, although mean sentiment is slightly higher for old-referencing documents (0.107) than for new-only documents (0.080), a Wilcoxon rank-sum test finds this difference is not statistically significant (W = 29,572, p = .25). We therefore cannot conclude that the original RAZR is viewed more favourably than the new one; on the available evidence, there is no reliable sentiment difference between nostalgia-driven and new-focused discussion.

```r
era_sentiment <- topic_source |>
  filter(!is.na(era))

cat("Sample sizes by era:\n")
```

```
## Sample sizes by era:
```

```r
print(table(era_sentiment$era))
```

```
##
##       new_only references_old
##           1698             39
```

```r
# Wilcoxon rank-sum (non-parametric, two groups)
if (length(unique(era_sentiment$era)) == 2 &&
    all(table(era_sentiment$era) > 5)) {
  era_test <- wilcox.test(sentiment ~ era, data = era_sentiment)
  print(era_test)
} else {
  cat("Not enough old-brand references for a reliable test.\n")
}
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  sentiment by era
## W = 29572, p-value = 0.2525
## alternative hypothesis: true location shift is not equal to 0
```

```r
# Mean sentiment by era
era_sentiment |>
  group_by(era) |>
  summarise(mean_sentiment = mean(sentiment, na.rm = TRUE),
            n = n(), .groups = "drop") |>
  knitr::kable(digits = 3, caption = "Sentiment: old-brand references vs new-only")
```

Table 7: Sentiment: old-brand references vs new-only

era	mean_sentiment	n
new_only	0.080	1698
references_old	0.107	39

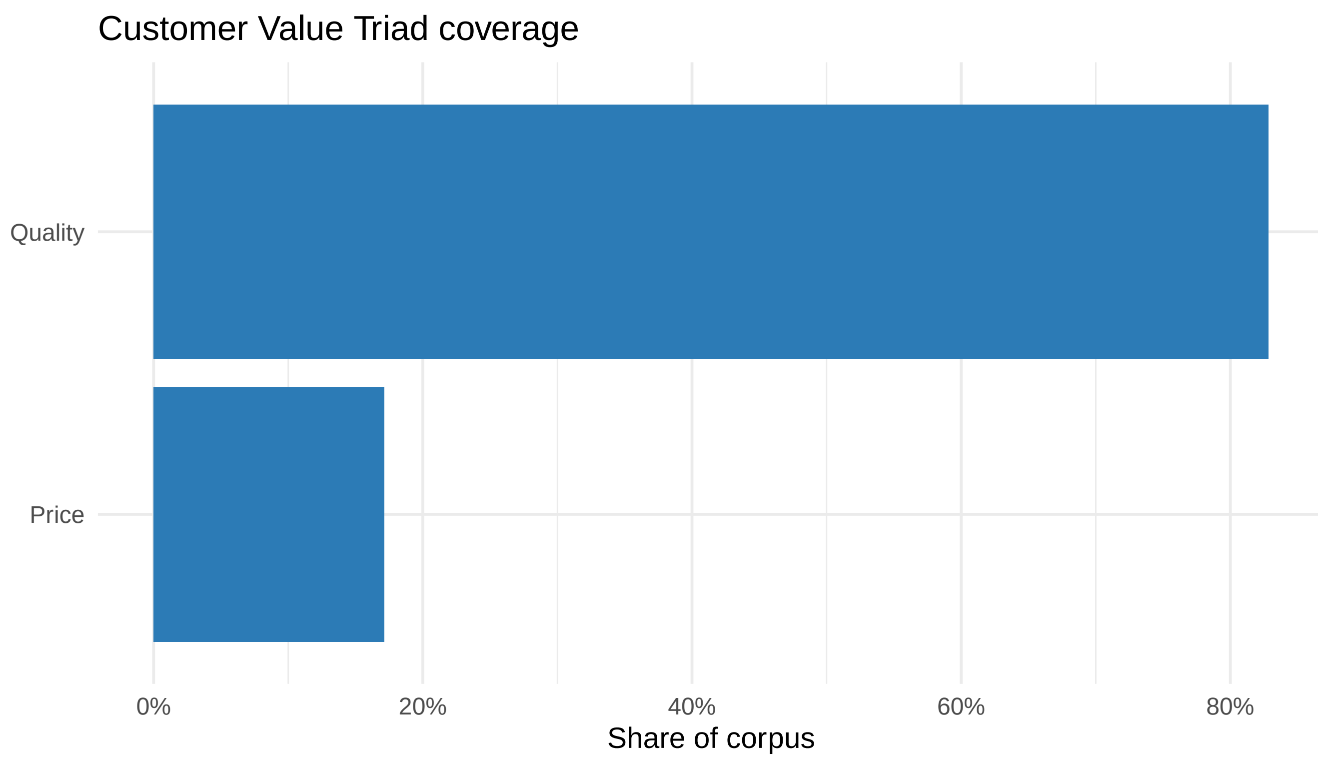
Customer Value Triad

To conclude, attention is given to the 'customer value triad' (quality, price, service), and what their share is of the corpus. What the chunk below shows is two major conclusions: the quality aspect easily dominates the corpus, and the service aspect is completely neglected. This may also speak to the appeal of either the RAZR or Motorola as a whole, and what is either of importance or relevance to their consumers. The complete absence of Service, is a meaningful finding in its own right, suggesting that after-sales support and service are simply not part of how this audience evaluates the RAZR.

```
triad_map <- data.frame(
  topic = paste0("topic", 1:7),
  dimension = c("Quality",   # Durability & Build Quality
                "Quality",   # Form Factor & Design
                "Quality",   # Competitor Comparison
                "Quality",   # Hardware & Specs
                "Price",     # Price & Purchase
                "Quality",   # Apps & Functionality
                "Quality"))  # Software & Support

triad_summary <- topic_counts |>
  mutate(topic = as.character(topic)) |>   # already "topic1", just coerce factor->char
  left_join(triad_map, by = "topic") |>
  group_by(dimension) |>
  summarise(total_prop = sum(prop), .groups = "drop")

ggplot(triad_summary, aes(x = reorder(dimension, total_prop), y = total_prop)) +
  geom_col(fill = "#2C7BB6") +
  coord_flip() +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Customer Value Triad coverage",
       x = NULL, y = "Share of corpus") +
  theme_minimal()
```




```r
knitr::kable(triad_summary, digits = 3,
             col.names = c("Value dimension", "Share of corpus"),
             caption = "Customer value triad reconstructed from topics")
```

Table 8: Customer value triad reconstructed from topics

Value dimension	Share of corpus
Price	0.172
Quality	0.828

Reflection

Nostalgia is a tool often used to aid brands revitalize previous successes, the RAZR's re-branding is no different. Using the data collected from YouTube and Reddit the RAZR's sentiment and prominence with regards to the seven outlined topics shows that Motorola has to an extent succeeded in cashing in on the nostalgia.

With positive sentiments around the price and design, it shows that consumers do not mind their pricing strategy and do very much enjoy the old design which was notorious at the time. But if motorola were to want to satisfy its customers further in and increase the middle sentiment what broader, then extra steps must still be taken to improve the software and apps functionality and support in comparison to its competitors, which are mainly Samsung and Apple.

However, for the lowest sentiment score of durability and build quality there may have an additional caveat which could be to Motorola's advantage. The field of foldable smartphones is quite a new one, where one phone iteration may already introduce a wide range of improvements, the Motorola RAZR and Samsung

Galaxy Z-Flip and Z-Fold are already examples of this. The build quality is still compromised to account for the innovative folding display, but is not per-se one specific to the RAZR. Meaning that if the path of innovation continues and Motorola works more on the software side of the equation, the RAZR re-brand may serve as a positive example for what one may look like.

The utilized methods may always have room for refinement as briefly discussed during the coding process. However, it is believed that the sources from which the data was collected as well as the methods which were used are of sufficient quality to extract clear, concise and reliable results for the purpose of this analysis. Though future iterations may work to experiment with broader sources of collection, as well as a more in depth analysis on the RAZR and its competitors for the sake of comparison.